

Brain of the circuit is Arduino Uno MCU board (BOARD1). A 16x2 LCD display (LCD1) is used for receiving and displaying the messages. Pins of LCD1, namely, RS, EN, D4, D5, D6 and D7 are connected to Arduino digital pins 12, 11, 5, 4, 3 and 2, respectively. Here, the LCD is used in 4-bit mode, which means only four data lines are required to display data.

When you want to control an equipment and show its status information, you need to send an SMS to GSM modem SIM900A (connected with CON2 and CON7). Arduino reads the message received by the GSM modem and sends commands to control the equipment as well as display its status on LCD1.

GSM modem sends the commands in text mode to Arduino using RS232 interface. CON7 is connected with connector DB9 of the GSM modem. RS232 voltage levels are at 12V, whereas both MCU input and output operate at 0V to +5V. Since RS232 is not compatible with the MCU, MAX232 (IC1) is utilised to enable communication between the GSM modem and Arduino Uno by converting RS232-level signals to TTL-level signals. Rx (DB9.3) and Tx (DB9.2) pins of the GSM modem are connected through IC1 at Tx and Rx pins of Arduino, respectively.

AT command is sent from Arduino to the GSM modem as the program starts up, and the response is received from the GSM modem after a very short period of time (not exceeding 500 microseconds), which is fast enough to detect incoming messages from the modem.

After receiving the command to turn on loads, the relay set performs the switching operation from normally-open to close the circuit. We have used one DC load (CON3) for the solenoid door-lock control and two AC loads (CON5 and

CON6) for AC light lamps control in our circuit.

Software

The code written in Arduino can communicate with the GSM modem using AT commands. AT commands are sent or received from the modem using serial communication functions provided by Arduino library.

Serial.begin(9600) function helps initialise the serial port with a baud rate of 9600.

Test Points

Test point	Details
TP1	5V
(Voltage measured with respect to ground)	

PARTS LIST

Semiconductors:	
BOARD1	- Arduino Uno
IC1	- MAX232
T1-T3	- BC547 npn transistor
Resistors (all 1/4-watt, $\pm 5\%$ carbon):	
R1-R3	- 1-kilo-ohm
PR1	- 10-kilo-ohm preset
Capacitors:	
C1, C3-C7	- 10 μ F, 16V electrolytic
C2	- 100nF, ceramic disk
Miscellaneous:	
LCD1	- 16x2 alphanumeric display
RL1-RL3	- 5V, 1C/O relay
CON1-CON7	- 2-pin connector terminal
	- GSM modem SIM900A
	- DC 12V/12V battery
	- 12V solenoid lock
	- 25W, 230V AC bulb

Serial.available() function is always called before reading a data byte from the serial port of Arduino. This function returns a positive value when data byte reception is complete; otherwise it returns a non-positive value. It serves the purpose of waiting till data byte reception is complete, so that there are no errors in reading the data byte.

Serial.read() function is used to read a data byte from the serial port of Arduino. It can return the data byte, which can then be stored in a variable or used for some condition check, among other things. The following statements show how the

data byte is read from the serial port and stored into a variable:

+ *CMGF AT* command is used to set SMS mode. Either text or protocol description unit (PDU) mode can be selected by assigning 1 or 0 in the command.

+ *CNMI AT* command is used to specify how newly-arrived SMS messages should be handled. It can be used to set the GSM/GPRS modem or mobile phone, either to route received SMS messages directly to the serial port or to save them in message storage and then notify the host device by sending a string through the serial port.

For example, to route the received message string to the serial port of the device, code written for this project uses *AT + CNMI = 2,2,0,0,0* command. Once an SMS is received, the GSM module sends a long string to Arduino board, which consists of many details regarding the message and the original text.

The code is written in such a way that it waits till any character is received through the serial port and once it starts receiving, checks for the occurrence of sixth ' ' and writes rest of the characters directly into the 16x2 LCD. Characters after the sixth ' ' in the received string is the original text and the rest of the characters are ignored in the code.

In this project we have used three home/office appliances for demonstration. These are solenoid-controlled door lock/unlock (CON3), light1 on/off (CON6) and light2 on/off (CON5).

Send *SOLO* as SMS to the SIM card number attached to the modem in your circuit. The door will open and the corresponding string *DOOR OPEN* will be displayed on the LCD.

Send *SOLOFF* as an SMS again; door will close and corresponding string *DOOR CLOSED* will be displayed on the LCD.

Send *LIGHTON* as an SMS; light will switch on and corresponding string *LIGHT 1 ON* will be displayed